Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

DESARROLLO DE APLICACIONES MÓVILES

Guía Práctica de Laboratorio N.º 09

Capacidades nativas con expect/actual en PharmaMobil

Dato	Detalle	Dato	Detalle
Asignatura	Desarrollo de Aplicaciones Móviles	Ciclo	VI · Semestre 2026-2
Unidad	2. Conectividad, CRUD REST y persistencia multiplataforma	Sesión	9 (Unidad 2, sesión 3)
Fecha	6 de octubre de 2026	Duración	2 horas teóricas + 2 horas prácticas
Proyecto	PharmaMobil (KMP) sobre PharmaSoft (API REST)	Modalidad	Laboratorio, trabajo en parejas

1. Ficha de la sesión según el sílabo

Toda la práctica que sigue desarrolla, sin añadidos ni recortes, el contenido que el sílabo asigna a esta sesión. Esta ficha permite verificar esa correspondencia.

Elemento del sílabo	Contenido
Asignatura	Desarrollo de Aplicaciones Móviles · EP Ingeniería de Sistemas · Facultad de Ingeniería y Arquitectura
Ciclo y plan	Ciclo VI · Plan de estudios 2024-1 · Año y semestre académico 2026-2
Créditos y horas	3 créditos · 2 horas teóricas y 2 horas prácticas por sesión
Prerrequisito	Desarrollo de Aplicaciones Distribuidas
Docente	Mg. Reyna Barreto Benjamin David
Nota mínima aprobatoria	13

1.1 Ubicación de la sesión

Elemento	Contenido
Unidad	Unidad 2. Conectividad, CRUD REST y persistencia multiplataforma
Sesión	Sesión 3 de la Unidad 2 (sesión 9 del curso) · 06-10-2026
Contenido del sílabo	«Mecanismo expect/actual, capacidades nativas por plataforma e interoperabilidad Kotlin-Swift.»
Actividad práctica	«Implementa una capacidad nativa usando expect/actual.» (2 horas)
Actividad autónoma	«Documenta diferencias por plataforma y evidencia iOS/Android.» (2 horas)

1.2 Competencias, resultado de aprendizaje y producto

Elemento	Contenido
Competencia específica	Ingeniería de Software: gestiona y desarrolla software de manera eficiente y efectiva, basándose en estándares internacionales de calidad. Dimensión 1.1 Programación y desarrollo de soluciones de software.
Competencia general	Carácter y aprendizaje autónomo. Dimensión 1.1 Firmeza de propósito, ejecución, dominio propio y mantener el esfuerzo.
Resultado de aprendizaje de la Unidad 2	Integra conectividad REST, autenticación, manejo de errores y persistencia local offline-first en una aplicación KMP.
Producto de la Unidad 2	Aplicación KMP con CRUD REST completo con Ktor, autenticación JWT y persistencia local offline-first.
Criterios de evaluación del producto U2	Consumo REST con Ktor · CRUD completo · Manejo de estados y errores · Autenticación JWT · Persistencia local y sincronización offline-first.
Estrategias metodológicas	Aprendizaje cooperativo y aprendizaje basado en problemas.

Dónde pesa esta sesión en la nota

La evaluación de sesiones aporta el 20 % de la nota final y la evaluación de productos el 70 %. El trabajo de hoy alimenta ambos componentes: la práctica se califica como evaluación de sesión y el código queda incorporado al Producto U2, que se sustenta el 27 de octubre de 2026.

2. Propósito de la práctica

Al finalizar esta práctica el estudiante implementa dos capacidades nativas en PharmaMobil mediante el mecanismo expect/actual y la inyección de implementaciones por plataforma, de modo que la interfaz y el dominio permanezcan en código común, y reconoce cómo el código Kotlin del proyecto se expone al lado Swift de la aplicación iOS.

Aporte al producto de la unidad

La Unidad 3 exige «uso de capacidades nativas o Firebase» como criterio del producto final. Esta práctica abre esa línea y prepara la sesión 10, donde el almacenamiento seguro del token JWT será otra capacidad distinta en cada plataforma.

2.1 Resultados esperados

- Función expect formatearSoles con su actual en androidMain y en iosMain.
- Precio de los productos mostrado con el formato de moneda del Perú en ambas plataformas.
- Interfaz Compartidor declarada en domain, con una implementación por plataforma.
- platformModule de Android y de iOS registrando sus respectivas implementaciones.
- Botón «Compartir» en el detalle del producto, funcionando en Android e iOS.
- Ninguna clase de presentation importando paquetes de Android ni de UIKit.

3. Requisitos previos

3.1 Conocimientos

- Organización de source sets en Kotlin Multiplatform: commonMain, androidMain e iosMain.
- Inyección de dependencias con Koin y el patrón de módulos del proyecto.
- Arquitectura Clean + MVVM tal como está implementada en PharmaMobil.
- CRUD REST sobre PharmaSoft, construido en las sesiones 7 y 8.

3.2 Entorno de trabajo

Esta es la sesión con mayor dependencia del entorno: el actual de iOS no compila sin macOS. Verifica el entorno antes de empezar.

Recurso	Detalle	Verificación
PharmaMobil	Rama con el CRUD de la sesión 8 integrado	La app lista productos en ambas plataformas
PharmaSoft	Backend en ejecución con datos de prueba	GET /api/v1/productos responde 200
Android Studio	Emulador operativo	La app arranca y muestra el listado
Xcode	Obligatorio para el target iOS	El proyecto iosApp compila y ejecuta
macOS	Necesario para compilar iosMain	Quien no lo tenga trabaja en pareja

Antes de empezar

Crea tu rama a partir de develop: git checkout -b feature/expect-actual-<apellido>. Cada integrante trabaja en su propia rama y se exigen commits distribuidos.

Abre AppModule.kt y los dos archivos PlatformModule: el mecanismo expect/actual ya existe en el proyecto desde la Unidad 1, pero ambos módulos están vacíos. Hoy dejarán de estarlo.

4. Marco conceptual de apoyo

Kotlin Multiplatform no promete compartirlo todo: promete compartir lo que tiene sentido compartir. El modelo de dominio, los casos de uso, el consumo REST y la interfaz Compose viven una sola vez en commonMain. Lo que depende del sistema operativo —formatos locales, permisos, sensores, compartir, almacenamiento seguro— necesita una implementación por plataforma.

El mecanismo expect/actual permite declarar en el código común qué debe existir, sin decir cómo se resuelve. Cada source set de plataforma aporta su actual, y el compilador exige que estén todos: si falta uno, el proyecto no compila. Ese es el valor del mecanismo frente a comprobaciones en tiempo de ejecución.

Concepto	Definición operativa
expect	Declaración sin cuerpo en commonMain. Dice qué debe existir.
actual	Implementación en un source set de plataforma. Debe coincidir en nombre, paquete y firma.
Biblioteca de plataforma	Kotlin/Native expone UIKit y Foundation como paquetes normales: platform.UIKit.UIDevice se importa como cualquier clase.
Framework compartido	El módulo shared se compila como un framework que Xcode importa con import Shared.
Sufijo Kt	Kotlin agrupa las funciones de nivel superior de un archivo en una clase con ese sufijo: MainViewControllerKt.
Prefijo do	Swift reserva los nombres que empiezan con init para los constructores, así que initKoinIos() se invoca como doInitKoinIos().

4.1 Dos estrategias para el mismo problema

Antes de escribir código conviene decidir qué mecanismo usar en cada caso. En esta práctica usaremos los dos.

Criterio	expect / actual	Interfaz + inyección
Quién exige la implementación	El compilador	El programador, al registrar en Koin
Dependencias externas	Difícil de recibir	Se inyectan en el constructor
Sustituir en una prueba	Complicado	Sencillo: se registra un doble

Criterio	expect / actual	Interfaz + inyección
Cuándo conviene	Utilidades sin estado, como un formateador	Capacidades con dependencias, como compartir

La combinación de esta práctica

El formateo de moneda se resuelve con expect/actual, porque es una función pura sin dependencias. Compartir se resuelve con una interfaz en domain y una implementación por plataforma, registradas dentro del expect val platformModule que el proyecto ya tiene. Así el compilador sigue exigiendo ambas implementaciones y las pruebas pueden sustituirlas.

5. Desarrollo de la práctica

Ejecuta los seis pasos en orden. No avances al siguiente hasta que el anterior compile en ambas plataformas.

Paso 1. Declarar la primera capacidad: formato de moneda

PharmaSoft envía el precio como número; mostrarlo correctamente es responsabilidad de cada plataforma. Crea el archivo en commonMain y deja que el compilador te reclame los actual.

```
// commonMain/platform/Formato.kt
package pe.edu.upeu.pharmamobil.platform

expect fun formatearSoles(valor: Double): String
```

Punto de control 1

Compila el proyecto. Debe fallar con un mensaje que indica que faltan las declaraciones actual para androidMain e iosMain. Ese error es la prueba de que el mecanismo funciona: lee el mensaje antes de continuar.

Paso 2. Escribir los dos actual

Ambos archivos deben declarar el mismo paquete y la misma firma que el expect.

```
// androidMain/platform/Formato.android.kt
package pe.edu.upeu.pharmamobil.platform

import java.text.NumberFormat
import java.util.Locale

actual fun formatearSoles(valor: Double): String =
    NumberFormat.getCurrencyInstance(Locale("es", "PE")).format(valor)
```

```
// iosMain/platform/Formato.ios.kt
package pe.edu.upeu.pharmamobil.platform

import platform.Foundation.*

actual fun formatearSoles(valor: Double): String {
    val formateador = NSNumberFormatter()
    formateador.numberStyle = NSNumberFormatterCurrencyStyle
    formateador.locale = NSLocale("es_PE")
    return formateador.stringFromNumber(NSNumber(valor))
        ?: "S/ $valor"
}
```

Aplica el formato donde corresponde: en el mapeo a ProductoUi de la capa de presentación, no dentro del composable ni en el modelo de dominio.

```
// presentation/producto/ProductoUi.kt
fun Producto.toUi() = ProductoUi(
    id = id,
    nombre = nombre,
    precio = formatearSoles(precio),
    stock = stock
)
```

Punto de control 2

El proyecto compila y el listado muestra el precio formateado. Compara el resultado en los dos dispositivos: los separadores y la posición del símbolo pueden diferir, y esa diferencia es precisamente lo que no podías resolver en código común.

Paso 3. Declarar la segunda capacidad como contrato del dominio

Compartir necesita algo que commonMain no conoce: un Context en Android y un controlador de vista en iOS. Por eso aquí conviene una interfaz, no un expect.

```
// commonMain/domain/platform/Compartidor.kt
package pe.edu.upeu.pharmamobil.domain.platform

interface Compartidor {
    fun compartir(texto: String)
}
```

El texto que se comparte se arma en código común, con el producto del dominio y el formateador del paso anterior:

```
// commonMain/domain/usecase/TextoParaCompartir.kt
fun Producto.comoTextoParaCompartir(): String =
    "$nombre — ${formatearSoles(precio)} · Stock: $stock"
```

Punto de control 3

La interfaz y la función de texto compilan en commonMain. Ninguna de las dos menciona Android ni iOS.

Paso 4. Implementar la capacidad en cada plataforma

```
// androidMain/platform/CompartidorAndroid.kt
class CompartidorAndroid(
    private val contexto: Context
) : Compartidor {

    override fun compartir(texto: String) {
        val envio = Intent(Intent.ACTION_SEND).apply {
            type = "text/plain"
            putExtra(Intent.EXTRA_TEXT, texto)
        }
        val selector = Intent.createChooser(envio, null).apply {
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        }
        contexto.startActivity(selector)
    }
}
```

```
// iosMain/platform/CompartidorIos.kt
class CompartidorIos : Compartidor {

    override fun compartir(texto: String) {
        val controlador = UIActivityViewController(
            activityItems = listOf(texto),
            applicationActivities = null
        )
        UIApplication.sharedApplication
            .keyWindow
            ?.rootViewController
            ?.presentViewController(controlador, true, null)
    }
}
```

Por qué FLAG_ACTIVITY_NEW_TASK

El Context que Koin entrega es el de la aplicación, no el de una Activity. Lanzar una Activity desde el contexto de aplicación exige esa bandera; sin ella, Android lanza una excepción en tiempo de ejecución.

Paso 5. Llenar el platformModule de cada plataforma

Los dos módulos del proyecto están vacíos desde la Unidad 1. Este es el momento para el que fueron creados.

```
// androidMain/di/PlatformModule.android.kt
actual val platformModule: Module = module {
    single<Compartidor> { CompartidorAndroid(androidContext()) }
}

// iosMain/di/PlatformModule.ios.kt
actual val platformModule: Module = module {
    single<Compartidor> { CompartidorIos() }
}
```

Inyecta el Compartidor en el ViewModel del detalle y expón la acción hacia la pantalla:

```
// presentation/detalle/DetalleProductoViewModel.kt
class DetalleProductoViewModel(
    private val repository: ProductoRepository,
    private val compartidor: Compartidor
) : ViewModel() {

    fun compartir(producto: Producto) {
        compartidor.compartir(producto.comoTextoParaCompartir())
    }
}
```

Punto de control 4

Koin resuelve el Compartidor en ambas plataformas. Si en Android aparece un error de dependencia no encontrada, revisa que initKoin siga recibiendo androidContext desde MainApplication.

Paso 6. Agregar el botón y verificar en ambas plataformas

1. Agrega un botón «Compartir» al detalle del producto, en código Compose común.
2. Conéctalo a la acción del ViewModel; el composable no debe conocer la implementación.
3. Ejecuta en el emulador de Android y comprueba que aparece el selector del sistema.
4. Ejecuta en el simulador de iOS y comprueba que aparece la hoja de compartir.
5. Verifica que el texto compartido incluya el precio con el formato de cada plataforma.
6. Captura ambas pantallas: son la evidencia de la práctica.

```
// presentation/detalle/DetalleProductoScreen.kt
Button(onClick = { onCompartir() }) {
    Icon(Icons.Default.Share, contentDescription = null)
    Spacer(Modifier.width(8.dp))
    Text("Compartir")
}
```

Punto de control 5

Busca en todo el paquete presentation las importaciones android. y platform.UIKit. No debe haber ninguna: si aparece alguna, la capacidad se filtró a la capa equivocada.

6. Interoperabilidad Kotlin-Swift en tu propio proyecto

PharmaMobil ya contiene dos archivos Swift que usan tu código Kotlin desde la Unidad 1. Ábrelos y obsérvalos con lo aprendido hoy.

Lo que ves en Swift	Por qué se llama así
KoinIosKt.doInitKoinIos()	Kotlin agrupa las funciones de nivel superior del archivo KoinIos.kt en la clase KoinIosKt. El prefijo do aparece porque Swift reserva los nombres que empiezan con init para los constructores.
MainViewControllerKt.MainViewController()	Mismo criterio: la función de nivel superior de MainViewController.kt queda dentro de una clase con sufijo Kt.
import Shared	El módulo shared se compila como framework; Shared es el nombre con el que Xcode lo importa.

6.1 Lo que cambia al cruzar

- Las funciones suspend se exponen como funciones async de Swift; para poder capturar errores deben anotarse con @Throws en Kotlin.
- Las sealed classes llegan como jerarquías de clases y pierden la exhaustividad: el switch de Swift siempre pedirá un default.
- Las data classes pierden copy y la desestructuración; se usan como clases normales.
- Los tipos anulables de Kotlin llegan como opcionales de Swift; los no anulables, como valores directos.

Consecuencia de diseño

Como la exhaustividad del when no cruza a Swift, la lógica de estados debe quedarse en Kotlin. Si alguien del equipo integra una pantalla SwiftUI, debe consumir un estado ya resuelto, no reconstruir la decisión en el otro lado.

7. Resolución de problemas frecuentes

Estos son los fallos que más aparecen al trabajar con código específico de plataforma.

Síntoma	Causa probable	Solución
Error de compilación: falta la declaración actual.	Solo se escribió el actual de una plataforma.	Crear el archivo en el otro source set con el mismo paquete.
El actual no se reconoce aunque existe.	El paquete declarado no coincide con el del expect.	Verificar la línea package en los tres archivos.
El actual no compila por la firma.	Un parámetro cambió de nombre o de tipo.	Copiar la firma del expect sin modificarla.

Síntoma	Causa probable	Solución
Excepción al compartir en Android.	Se lanza una Activity desde el contexto de aplicación.	Agregar FLAG_ACTIVITY_NEW_TASK al Intent del selector.
Koin no encuentra el Compartidor.	No se registró en el platformModule correspondiente.	Declarar el single en ambos PlatformModule.
No se encuentra androidContext().	Falta la dependencia koin-android o el registro en MainApplication.	Revisar initKoin y la declaración de la aplicación en el manifiesto.
No se puede compilar iosMain.	Se está trabajando en Windows o Linux.	Compilar el target de iOS en una máquina con macOS y Xcode.
La hoja de compartir no aparece en iOS.	El controlador raíz es nulo en ese momento.	Obtenerlo desde la ventana activa y validar la nulabilidad.

8. Entregable de la práctica

Al cierre de la sesión, cada pareja entrega a través del aula virtual:

- Enlace a la rama feature/expect-actual-<apellido> de cada integrante, con al menos tres commits propios.
- Captura del listado con el precio formateado, en Android y en iOS.
- Captura del selector de compartir de Android y de la hoja de compartir de iOS.
- Captura del mensaje de error de compilación obtenido en el punto de control 1.
- README actualizado con la sección «Capacidades nativas» y los archivos de cada plataforma.

9. Lista de cotejo de evaluación

Cada criterio vale dos puntos. El puntaje máximo de la práctica es veinte puntos.

N.º	Criterio observable	Cumple (Sí / No)
1	Existe una declaración expect en commonMain con su actual en androidMain y en iosMain.	
2	Los tres archivos declaran el mismo paquete y la misma firma.	
3	El precio se muestra formateado como moneda en ambas plataformas.	
4	El formato se aplica en la capa de presentación, no en el dominio ni dentro del composable.	
5	La interfaz Compartidor está declarada en domain y no menciona ninguna plataforma.	
6	Existe una implementación de Compartidor por plataforma, con sus APIs nativas.	
7	Ambos platformModule registran su implementación y Koin la resuelve.	
8	El botón Compartir funciona en Android y en iOS, con el texto armado en código común.	

N.º	Criterio observable	Cumple (Sí / No)
9	Ninguna clase de presentation importa paquetes de android. ni de platform.UIKit.	
10	Las capturas evidencian la misma funcionalidad en las dos plataformas.	

Actividad autónoma asociada

Documenta diferencias por plataforma y evidencia iOS/Android, según la ficha de actividad autónoma de la sesión 9. Entrega hasta el lunes 12 de octubre de 2026 a las 23:59.

10. Referencias

10.1 Referencias básicas del sílabo

- JetBrains. (2025b). Kotlin Multiplatform documentation. https://kotlinlang.org/docs/multiplatform.html
- JetBrains. (2025b). Expected and actual declarations. https://kotlinlang.org/docs/multiplatform-expect-actual.html
- JetBrains. (2025b). Interoperability with Swift/Objective-C. https://kotlinlang.org/docs/native-objc-interop.html
- JetBrains. (2025b). Kotlin/Native platform libraries. https://kotlinlang.org/docs/native-platform-libs.html
- Hogan, J. (2024). Mastering Kotlin Multiplatform: Build Cross-Platform Apps for Android, iOS, Web, and Desktop. Independently Published. ISBN 979-8-3461-9721-8.
- Touchlab. (2025). Kotlin Multiplatform resources. https://touchlab.co/
- JetBrains. (2025a). Compose Multiplatform. https://www.jetbrains.com/compose-multiplatform/

10.2 Recursos del curso

- PharmaMobil, aplicación del curso. https://github.com/dreyna/pharmaMobil
- PharmaSoft, backend del curso. https://github.com/dreyna/pharmaSoft